

Project Title: Electricity Bill Calculator

Student Name: SRI SELSIYA R

Registration No: 26MEI10009

Repo Link: <https://github.com/sri26mei10009-spec/Vityarthi-project.git>

1. Introduction

This project is a Python-based command-line application that calculates electricity bills. It takes the number of units a person has consumed and calculates the total cost using different pricing slabs. The code is organized into multiple files to keep it clean, and uses basic Python features like lists, loops, and conditional statements to work

2. Problem Statement

Many people find it hard to understand how their electricity bills are calculated because the price changes depending on how much they use. Doing this math manually is confusing and many can make mistakes. This project solves that by automatically doing the math and showing users exactly how their money is split across different price slabs.

4. Functional Requirements

The application includes the following working features:

- **Interactive Menu:** A simple menu that lets the user press '1' to calculate a bill or '2' to exit safely.
- **Data Input:** The program easily accepts numbers (including decimals) for the electricity units consumed.
- **Tiered Calculation:** It splits the units into different slabs (e.g., first 100 units at one price, next 100 at a higher price) using if-elif-else logic.
- **Detailed Output:** It prints the final total cost and a list showing exactly how much was charged in each individual slab.

5. Non-functional Requirements

The system is built with these quality standards in mind:

- **Usability:** The text prompts are clear and tell the user exactly what to type.
- **Error Handling:** If a user types letters instead of numbers, the program shows an error message instead of crashing.
- **Maintainability:** The code is split into 5 separate files, making it very easy to read and update later.
- **Performance:** The math is simple, so the calculation happens instantly.

6. System Architecture

The project uses a modular design divided into a main script and a program folder.

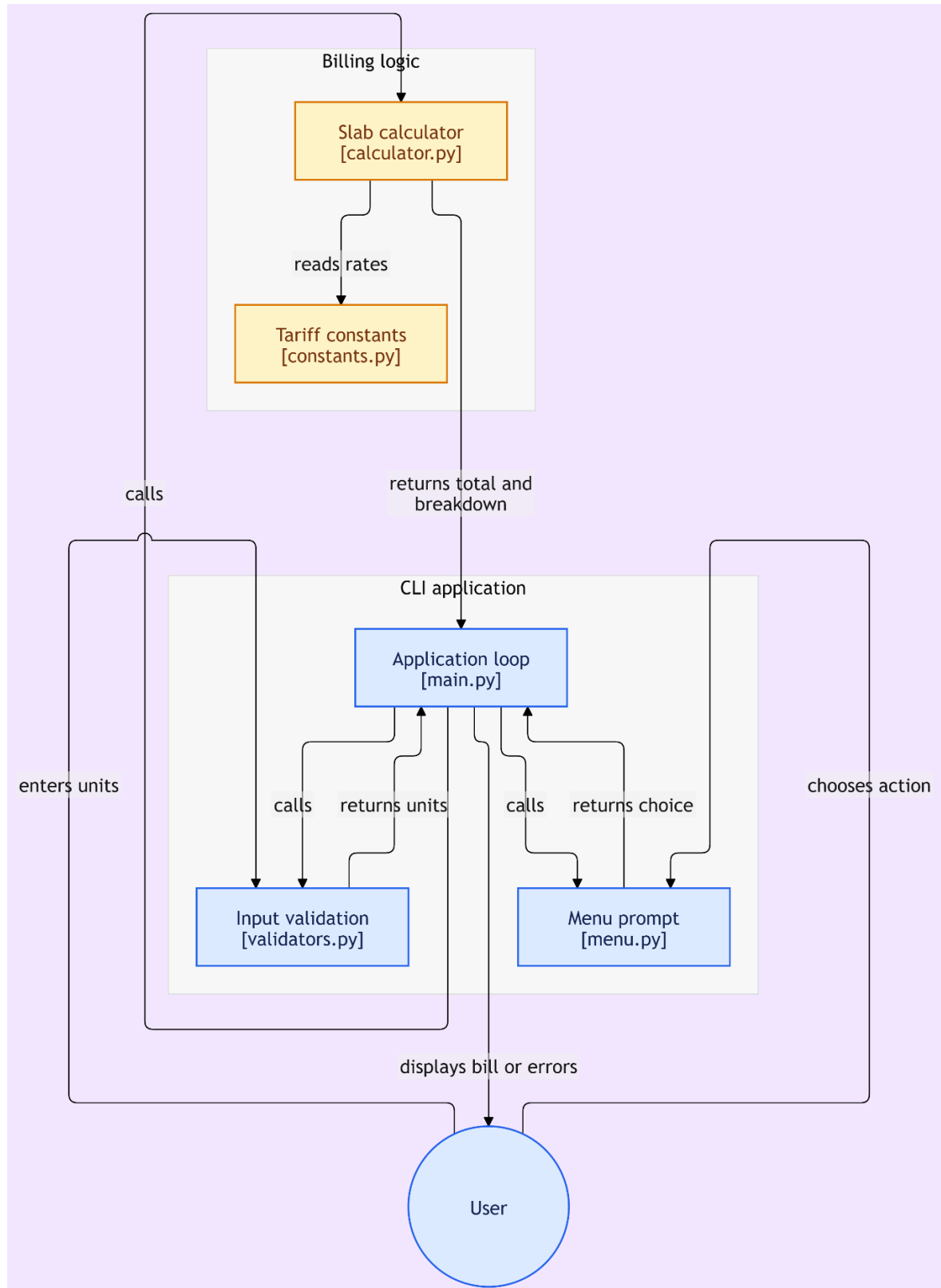
- main.py is the boss; it runs the menu loop.
- When the user wants to calculate something, main.py talks to menu.py for the visuals, validators.py to check the input, and calculator.py to do the actual math.
- All the prices are kept safely in constants.py.

Directory structure:

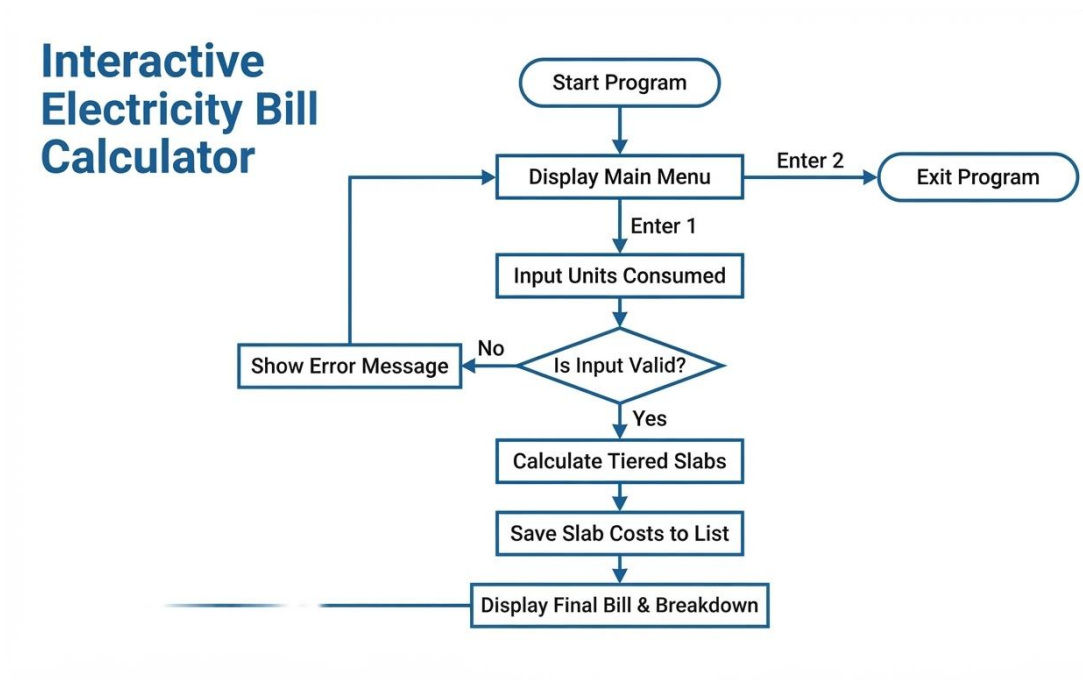
```
└─ sri26mei10009-spec-vityarthi-project/  
  └─ README.md  
  └─ main.py  
  └─ requirements.txt  
  └─ statement.md  
  └─ program/  
    └─ __init__.py  
    └─ calculator.py  
    └─ constants.py  
    └─ menu.py  
    └─ validators.py
```

7. Design Diagrams

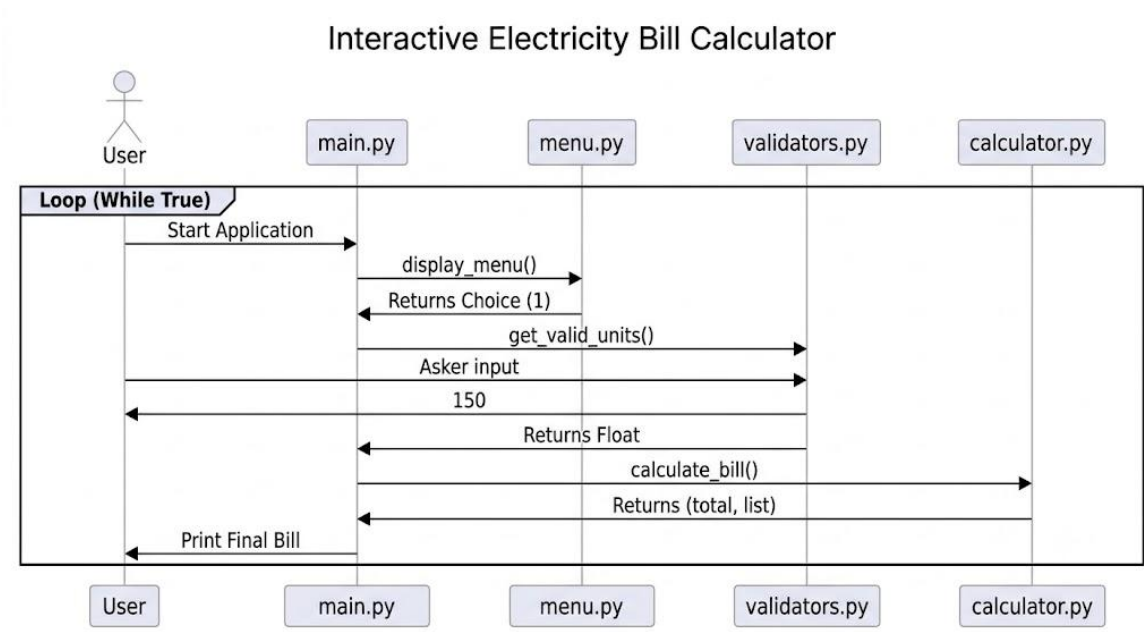
- **Use Case Diagram:** The User interacts with three main features: View Menu, Input Units, and View Bill Breakdown.



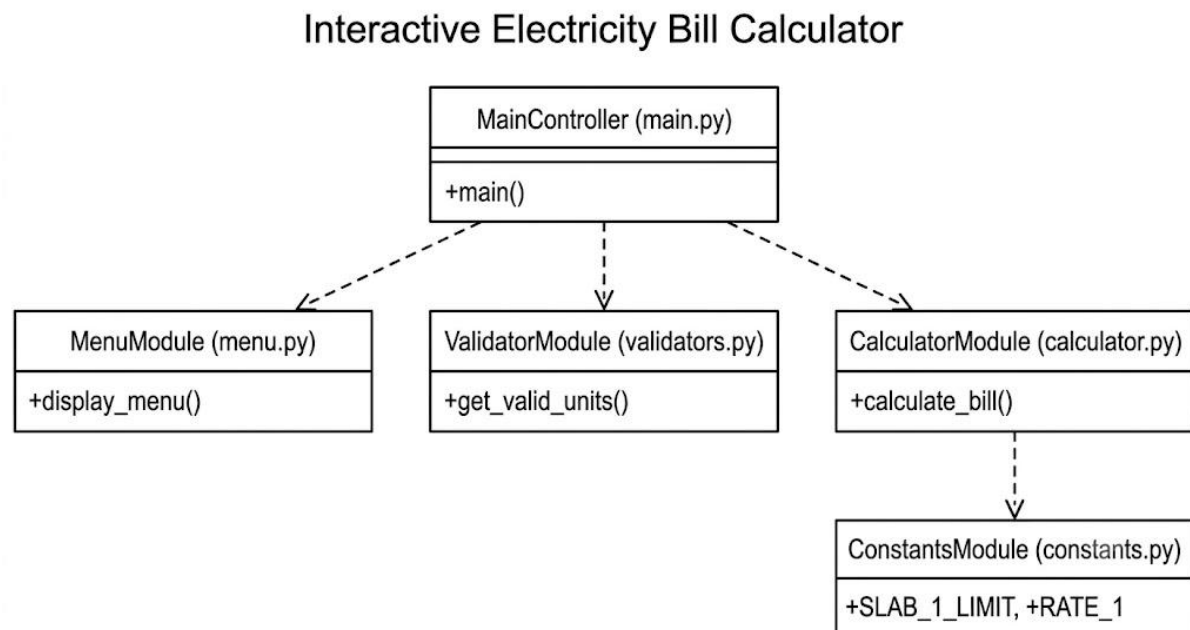
• **Workflow Diagram:**



• **Sequence Diagram:**



- **Class Diagram:**



8. Design Decisions & Rationale

- **Using Multiple Files:** Splitting the code makes it easier to fix bugs. If the electricity prices change, we only have to update constants.py without touching the math logic.
- **Using Lists for Cost:** Instead of just adding numbers together, the program appends the cost of each slab into a Python list. This makes it incredibly easy to print the step-by-step breakdown for the user.
- **Using a While Loop:** Wrapping the whole program in a while True loop means the user can calculate bills for five different houses in a row without having to restart the application every time.

9. Implementation Details

The project was written in Python 3 with the following files:

- **constants.py:** Holds our static numbers (like SLAB_1_LIMIT = 100).
- **validators.py:** Uses a try-except block to make sure the user types a real number and not something like "abc".
- **calculator.py:** Contains the main if-elif-else logic. It checks how big the input is, calculates the cost for that tier, saves it to a list, and returns the total sum.
- **main.py:** Connects everything and prints the final output to the screen.

10. Screenshots / Results

(Note for your PDF: Take actual screenshots of your black command prompt screen and paste them here)

- **Screenshot 1:** The menu and a normal calculation for 150 units

```
=====
      ELECTRICITY BILL CALCULATOR
=====

-----
1. Enter to calculate bill
2. Exit
Enter your choice (1 or 2): 1

Enter the number of units consumed: 150

--- Bill Summary ---
Cost breakdown per slab: [500, 350.0]
Total Electricity Bill: Rs 850.0

-----
1. Enter to calculate bill
2. Exit
Enter your choice (1 or 2): |
```

- **Screenshot 2:** an error when you type letters instead of numbers.

```
=====
ELECTRICITY BILL CALCULATOR
=====

-----
1. Enter to calculate bill
2. Exit
Enter your choice (1 or 2): 1

Enter the number of units consumed: fsfa
Error: Invalid input! Please enter a valid numerical value.

-----
1. Enter to calculate bill
2. Exit
Enter your choice (1 or 2): |
```

11. Testing Approach

To make sure the program works, three types of tests were done:

- **Normal Testing:** Typed in standard numbers 90 to make sure the math was correct.

```
-----
1. Enter to calculate bill
2. Exit
Enter your choice (1 or 2): 1

Enter the number of units consumed: 90

--- Bill Summary ---
Cost breakdown per slab: [450.0]
Total Electricity Bill: Rs 450.0
```

- **Boundary Testing:** Typed in exactly 100 and exactly 200 to make sure the program knew exactly when to switch to the next slab rate

```
-----
1. Enter to calculate bill
2. Exit
Enter your choice (1 or 2): 1

Enter the number of units consumed: 100

--- Bill Summary ---
Cost breakdown per slab: [500.0]
Total Electricity Bill: Rs 500.0
```

```
-----  
1. Enter to calculate bill  
2. Exit  
Enter your choice (1 or 2): 1  
  
Enter the number of units consumed: 200  
  
--- Bill Summary ---  
Cost breakdown per slab: [500, 700.0]  
Total Electricity Bill: Rs 1200.0
```

- **Negative Testing:** Intentionally typed in negative numbers (-50)

```
-----  
1. Enter to calculate bill  
2. Exit  
Enter your choice (1 or 2): 1  
  
Enter the number of units consumed: -50  
Error: Units cannot be negative. Please try again.
```

12. Challenges Faced

The hardest part of writing this code was figuring out the math for the "leftover" units when moving from one slab to the next. At first, the code was multiplying the higher price by *all* the units instead of just the extra ones. This was fixed by carefully subtracting the previous slab's limit (e.g., units - 100) before multiplying by the new rate.

13. Learnings & Key Takeaways

Building this project was a great way to practice real-world coding. I learned how to separate different parts of a program into their own files to keep things organized. I also learned that you can never trust what a user will type, so building strong error-handling (try-except blocks) is absolutely necessary to keep software from breaking.

14. Future Enhancements

If I had more time to improve the project, I would:

- Build a Graphical User Interface (GUI) with buttons instead of a text-based terminal.
- Let users save their past bills to a text file so they can track their electricity usage over the whole year.

15. References

- Python Official Documentation for learning lists and error handling.
- VITyarthi Platform - General Project Instructions & Submission Guidelines PDF.